Software Requirements Specification

For

Unbound: Fest Management Platform

Institution: MIT Academy of Engineering (MITAOE)

Project Goal: To architect a centralized, high-concurrency digital ecosystem for the lifecycle management of college fests, ensuring absolute data precision through robust Role-Based Access Control and automated registration flows.

Document Version:	2.2 (Technical Baseline)
Infrastructure:	AWS Fargate & PostgreSQL
Operational Capacity:	500 Concurrent user
Security Model:	Role-Based Access Control (RBAC)

Contents

1 Introduction **2**
1.1 Purpose and Engineering Vision 2
1.2 Revised User Hierarchy 2

2 System Architecture and Tech Stack **3**
2.1 Technical Stack Matrix 3

3 Functional Requirements (Business Logic) **4**
3.1 Role-Based Access Control (RBAC) 4
3.2 Event Orchestration Logic 4
3.3 Team Formation and Member Mapping 4

4 System Services and Integrations **5**
4.1 Payment Gateway (Razorpay) 5
4.2 Automated Notifications (Spring Mail) 5
4.3 Data Relational Mapping (ORM) 5

5 Use Case Scenarios: Student Interaction **6**
5.1 Use Case 1: Individual Registration (Solo Event) 6
5.2 Use Case 2: Join a Team via Code 6

6 Use Case Scenarios: Management Interaction **7**
6.1 Use Case 3: Create New Event 7
6.2 Use Case 4: Global System Audit 7

7 Non-Functional Requirements (Technical Benchmarks) **8**
7.1 Technical Stability and Performance 8
7.2 Security and Integrity 8

8 Conclusion **9**

1 Introduction

The Unbound project is a strategic technical response to the fragmented administrative landscape currently governing technical fests at MITAOE. It replaces inefficient, manual workflows with a centralized, professional-grade platform designed for high-throughput registration.

1.1 Purpose and Engineering Vision

This SRS establishes a definitive technical framework for the development of "Unbound". The platform is designed to handle the unique relational hierarchy of a university ecosystem, providing a single, high-performance command center for event orchestration.

1.2 Revised User Hierarchy

The system enforces a strict hierarchical access model to maintain institutional integrity:

User Role	Primary Access Right	Core Responsibility
Super Admin	Global System Privileges	Platform-wide auditing and institutional configuration.
College Admin	College-Level Privileges	Oversight of all club fests within a specific college entity.
Club Coordinator	Moderator Privileges	Event deployment, registration tracking, and participant management.
Student	Standard User Privileges	Event discovery, Solo registration, and Team formation.

Table 1: Refined User Access and Responsibility Matrix

2 System Architecture and Tech Stack

Unbound utilizes a decoupled Three-Tier Architecture. This ensures that the presentation, business logic, and data persistence layers can be scaled independently during peak registration surges.

2.1 Technical Stack Matrix

Component	Technology	Version	Engineering Justification
Language	Java	17.0 (LTS)	Enterprise stability and long-term support.
Backend Framework	Spring Boot	3.5.11	High-throughput, multi-threaded request processing.
Frontend Library	React	18.x	Modular UI components for reusable event dashboards.
Database	PostgreSQL	15.17	Relational ACID compliance for registration integrity.
Infrastructure	AWS Fargate	Latest	Serverless container orchestration for auto-scaling.
Payments	Razorpay SDK	1.4.3	Native Java integration for secure payment processing.

Table 2: Consolidated Technical Stack and Versioning

3 Functional Requirements (Business Logic)

3.1 Role-Based Access Control (RBAC)

The platform implements a rigid RBAC model at the Spring Security filter-chain level. Access is not generic; every request is validated against a signed JWT.

- **Authorization Scopes:** Specific permissions for ‘ROLE$_SUPER_ADMIN$‘, ‘$ROLE_COLLEGE_ADMIN$‘, ‘$ROLE_COORDINATOR$‘, and‘$ROLE_STUDE$

- **Stateless Session:** JWT tokens are utilized to handle high concurrency without server-side session overhead.

3.2 Event Orchestration Logic

Organizers define event parameters without the use of cloning templates, ensuring each event is a unique database entity:

- **Attribute Definition:** Name, Venue, Category (Solo/Team), Date, and Entry Fees.
- **Operational Caps:** Strict enforcement of maximum registration limits to prevent infrastructure over-saturation.

3.3 Team Formation and Member Mapping

The system automates the most complex part of fest management—relational mapping.

- **Solo Registration:** Direct ‘Student <-> Event‘ mapping.
- **Team Creation:** A Leader initiates a team; the system generates a unique alphanumeric Team Code.
- **Member Join:** Participants enter the Team Code; the system validates the code and slot availability before appending the user to the team entity.

4 System Services and Integrations

4.1 Payment Gateway (Razorpay)

Secure financial transactions are integrated directly into the registration flow.

- **Webhook Synchronization:** Automated updates to registration status once payment success is confirmed.
- **Transaction Integrity:** Atomic database commits ensure a student is only marked "Registered" after the payment ID is logged.

4.2 Automated Notifications (Spring Mail)

Real-time communication is handled via SMTP integration to ensure users remain informed throughout the lifecycle.

- **Registration Alerts:** Automated confirmation emails containing event details and payment receipts.
- **Team Coordination:** Sending unique Team Codes to leaders for distribution to their members.

4.3 Data Relational Mapping (ORM)

Using Spring Data JPA, the system manages complex relationships between Students, Teams, and Events in PostgreSQL, ensuring that cascading deletions or updates do not orphan participant records.

5 Use Case Scenarios: Student Interaction

5.1 Use Case 1: Individual Registration (Solo Event)

Actor: Student
Precondition: Student is logged in and event has available slots.
Flow: Student navigates to Event Dashboard → Selects "Register Now" → System triggers Razorpay payment modal → Payment successful → System generates registration record → Trigger confirmation email.
Postcondition: Student is registered; slot count is reduced.
Exception: 409 Conflict if student is already registered.

5.2 Use Case 2: Join a Team via Code

Actor: Student (Team Member)
Precondition: Student has received a valid Team Code from a Leader.
Flow: Select Team Event → Click "Join Team" → Enter Alphanumeric Team Code → System validates Code/Team Status → System maps Student to Team ID → Registration confirmed.
Postcondition: Student is part of the team record in the database.
Exception: 404 Not Found if Team Code is invalid.

6 Use Case Scenarios: Management Interaction

6.1 Use Case 3: Create New Event

Actor: Club Coordinator
Precondition: Authenticated with Moderator privileges.
Flow: Enter Event Details (Fees, Team Size, Category) → Submit to System → Data enters "Pending Review" status → College Admin notified.
Postcondition: Event metadata stored and awaiting activation.

6.2 Use Case 4: Global System Audit

Actor: Super Admin
Precondition: Authenticated with Global privileges.
Flow: Access Administrative Command Center → View Cross-Club Registration Metrics → Export consolidated reports.
Postcondition: Accurate reporting of institutional engagement.

7 Non-Functional Requirements (Technical Benchmarks)

7.1 Technical Stability and Performance

- **Concurrency Handling:** Stable performance at 500 concurrent users using AWS Fargate serverless containers.
- **Response Latency:** P95 latency for registration and payment validation must remain below 300ms.

7.2 Security and Integrity

- **Data Security:** Mandatory BCrypt hashing for all credentials and HTTPS/TLS 1.3 for data in transit.
- **Relational Integrity:** Zero-duplication logic enforced via database-level unique constraints in PostgreSQL.

8 Conclusion

Unbound is designed as a robust, high-performance ecosystem for MITAOE. By prioritizing a strict Role-Based Access Control model, seamless payment and mail integrations, and clear Solo vs. Team registration scenarios, the platform effectively eliminates the inefficiencies of manual administrative tools.

Through the use of a Decoupled Three-Tier Architecture and AWS cloud orchestration, the system ensures transaction integrity and institutional oversight for all technical fests. This document stands as the final technical agreement and operational manual for the production-ready release of the Unbound platform.